myKaria: a production AI product built and run by one developer

Oluwatobi Lukan | Hamburg, Germany | Full-stack engineer, AI application pipelines

myKaria turns a job ad into a tailored CV and cover letter that a candidate can actually defend in the interview. I designed it, built it and run it: database, backend, frontend, AI pipeline, billing and deployment. Around 200 commits, 68 migrations, 23 document templates, live in production.

THE PROBLEM, STATED PRECISELY

AI CV tools optimise for interview invitations, so they exaggerate. The candidate gets the interview and loses it, because the document claims things they cannot back up. Recruiters now discount generated CVs by default. So defensibility is not a writing problem, it is a ranking and policy problem: which skills genuinely match, at what level they can honestly be claimed, which ones to feature or omit, and how to cover missing requirements with real adjacent experience rather than invention.

THREE DECISIONS THAT SHAPED THE BUILD

- **One matching call, not three.** The obvious pipeline is match, plan, write. I collapsed it to a single indexed matching call whose result feeds both documents. That removed a third of the latency and removed a class of bug where the CV and the letter disagreed about the same skill.
- **The model judges, the code enforces policy.** Anything that must hold every time, such as a beginner skill never appearing as expert, is implemented in tested code rather than requested in a prompt. Truncated model output is treated as a correctness failure, because an empty skill list used to let anything through.
- **A modular monolith.** At one developer, splitting services adds operational cost and buys nothing. The useful boundaries are module boundaries: 33 route modules, domain services, their own tests.

EVIDENCE

- 148 unit tests in the public slice covering prompt assembly and the matching pipeline end to end, including failure paths
- 274 test and spec files across the full repository
- Provider-agnostic LLM layer: OpenAI, Anthropic, Ollama, OpenRouter and any OpenAI-compatible endpoint, with retry, fallback and per-call cost resolution
- Multi-tenant mechanics: JWT auth with email verification, tier entitlements, Stripe billing, data isolation, retention and deletion jobs
- Rendering pipeline: Handlebars templates to HTML to PDF through Puppeteer
- Docker Compose environments, Jenkins pipeline, Nginx-backed deployment

WHAT WENT RIGHT, AND WHAT I GOT WRONG

The engineering held. The pipeline is deterministic where it needs to be, and the tests caught real regressions in prompt assembly before users did.

What I got wrong was sequencing. I built breadth, an admin console, a CMS, a translator, before proving anyone could find the product. It is live and it has no meaningful traction, and the reason is distribution, not features. Starting again, I would ship one narrow flow to real users first and earn the right to build the rest.

WHAT THIS SAYS ABOUT ME AS AN ENGINEER

I can take an ambiguous product idea to a running system: schema design, provider abstraction, prompt engineering with guardrails, billing, deployment, and the unglamorous parts such as retention jobs and idempotency. I know where AI belongs in a system, and more usefully, where it does not.

Product mykaria.com | Code github.com/ztobs/mykaria-showcase | Architecture notes in *docs/architecture.md* | Contact: tobilukan@gmail.com